

Devoir surveillé terminal

Durée 2h

*Aucun document, outil de calcul ou de communication autorisé. Sauf indications contraires, la gestion d'erreur **ne doit pas** être réalisée et les inclusions ne doivent pas être spécifiées.
Toute réponse doit être justifiée.*

Bien qu'indépendants, les exercices de ce sujet suivent la modélisation suivante.

Nous souhaitons développer une simulation d'arène sur laquelle combattent des robots. L'arène est découpée en cases : sur chacune, il ne peut y avoir qu'un robot, un obstacle ou rien. Cette simulation correspond en fait à trois applications : le *synchroniseur*, le *serveur* et les *clients* (correspondant aux robots).

La simulation est découpée en tours pendant lesquels chaque robot ne peut faire qu'une action : attaquer un robot situé sur une case adjacente (haut, bas, droite et gauche), attendre ou se déplacer (sur une case vide). Si l'action choisie n'est pas possible (si un robot s'est placé sur la case vide sur laquelle le robot voulait se déplacer ou si le robot attaqué s'est déplacé entre temps), le robot perd alors son action et reste sur sa position.

Le synchroniseur crée l'arène puis met en place les différents outils IPC dont les clés sont fixées par les constantes `CLE_SEGMENT`, `CLE_FILE` et `CLE_SEMAPHORE` puis les initialise. Il affiche ensuite l'état de la simulation à l'aide d'une interface *ncurses* : une première fenêtre permet d'afficher les logs (lorsqu'un robot se connecte, est détruit ou réalise une action) et l'état de l'arène est affiché dans une deuxième fenêtre. C'est le synchroniseur qui est en charge de gérer les tours. Il prend en arguments les dimensions de l'arène (largeur et hauteur) et le nombre maximum de robots.

Le serveur prend en argument un numéro de port d'écoute TCP. Lorsqu'un client se connecte, le serveur crée un processus fils qui est en charge de la connexion établie. Il envoie ensuite un message via la file de messages au synchroniseur pour l'avertir de l'arrivée du robot. Le processus fils communique ensuite avec le client : envoi de l'état de l'arène, demande d'ordres, attaques éventuelles et réception des actions du robot. Il envoie les actions au synchroniseur via la file de messages une fois l'arène mise à jour.

Le client correspond à un patron permettant ainsi de simuler des robots avec des comportements différents et peut être exécuté plusieurs fois. C'est une application *ncurses* qui prend en arguments l'adresse IP du serveur et son numéro de port.

Exercice 1 (Le segment de mémoire partagée (4 points))

Le segment de mémoire partagée contient toutes les données accessibles à la fois par le synchroniseur, le serveur et ses processus fils. Il contient les dimensions de l'arène, l'arène en elle-même (les cases), le nombre maximum de robots et les points de vie de chacun (à 10 au départ, décrémenté de 1 à chaque attaque subie).

1°) Précisez le contenu du segment de mémoire partagée (données + types) puis proposez une structure `segment_t` permettant de le mapper tel que vu en CM/TD (elle contiendra notamment un champ pour l'identifiant du segment et un pour son adresse mémoire).

2°) Donnez le code de la fonction `recup_segment` permettant à un fils du serveur de récupérer le contenu du segment de mémoire partagée. Elle ne prend aucun paramètre et retourne un pointeur sur un `segment_t` qui permet de mapper le segment.

Exercice 2 (Les problèmes de synchronisation (7 points))

1°) Nous souhaitons que les fils du serveur attendent passivement que le tour commence : c'est le synchroniseur qui le déclenche. Proposez une solution en utilisant les sémaphores (décrivez les sémaphores utilisés, leur initialisation, ainsi que les opérations nécessaires).

2°) Que se passe-t-il si un robot se connecte entre temps ? Votre solution fonctionne-t-elle ? Expliquez et proposez une solution si ce n'est pas le cas (toujours à l'aide de sémaphores).

3°) Expliquez les problèmes de synchronisation qui peuvent survenir lors des actions des robots. Proposez une solution à base de sémaphores pour les résoudre en limitant leur nombre sans empêcher les exécutions parallèles.

4°) Donnez le code permettant au synchroniseur de créer le tableau de sémaphores en fonction de vos réponses aux questions précédentes et de l'initialiser. Si le tableau existe, il est supprimé, sauf si le nombre de sémaphores correspond (dans ce cas, on initialise simplement les sémaphores).

Exercice 3 (Le serveur (5 points))

Le serveur récupère le nombre de robots maximum via le segment de mémoire partagée. Lorsqu'une demande de connexion est reçue, le serveur vérifie si le nombre maximum de robots est déjà atteint. Si c'est le cas, il envoie un code d'erreur (un `unsigned char`) au client. Sinon, il envoie un code de succès (un `unsigned int`) puis crée un processus fils.

1°) Sachant qu'un robot peut s'arrêter à tout moment, proposez une solution permettant au serveur de connaître le nombre exact de clients connectés.

2°) Donnez la portion de code du serveur qui attend une connexion (en supposant la socket déjà créée et nommée), envoie le code de succès/d'erreur au client (en fonction du nombre actuel de clients) puis crée le fils associé (si le nombre maximum de clients n'est pas atteint).

- La procédure `fils` correspond au code d'un fils : vous donnerez sa signature en expliquant chaque paramètre.
- Chaque variable devra être déclarée (et initialisée si besoin).
- La fonction `getNumero()` retourne le numéro du nouveau client (entre 0 et le nombre maximum de clients moins 1) ou -1 si le nombre maximum de clients est atteint.

3°) Donnez toutes les modifications de code (variables nécessaires comprises) pour afficher sur le terminal l'adresse du client (IP+port) qui a demandé une connexion.

Exercice 4 (Les communications avec le synchroniseur (4 points))

Lorsque des robots se déplacent, les fils du serveur mettent à jour directement l'arène dans le segment de mémoire partagée (mise-à-jour des cases ou des points de vie des robots). Pour qu'il soit averti de ces actions, des messages sont envoyés au synchroniseur soit par le serveur (lors des connexions), soit par les fils du serveur.

1°) Proposez une/des structures représentant les messages envoyés dans la file de messages.

2°) Donnez les deux portions de code du serveur permettant de récupérer la file et d'envoyer un message pour avertir le synchroniseur de la connexion d'un nouveau client.

3°) Rappelez les problèmes rencontrés avec une file pour la récupération de messages de longueurs différentes. Proposez une solution.

Annexes : quelques en-têtes de fonctions C

```

int      accept(int sockfd, struct sockaddr *adresseClient, socklen_t *taille);
unsigned int alarm(unsigned int nb_secondes);
int      atexit(void (*procedure)(void));
int      bind(int fd, struct sockaddr *adresse, socklen_t longueur);
int      close(int fd);
int      connect(int sockfd, struct sockaddr *adresseServeur, socklen_t taille);
int      execve(const char *fichier, char *const argv[], char *const envp[]);
void     exit(int statut);
int      fcntl(int fd, int commande, ...);
pid_t    fork();
key_t    ftok(char *chemin, int proj_id);
pid_t    getpid();
pid_t    getppid();
int      getpeername(int sockfd, struct sockaddr *adresse, socklen_t *taille);
int      getsockname(int sockfd, struct sockaddr *adresse, socklen_t *taille);
uint32_t htonl(uint32_t hote_long);
uint16_t htons(uint16_t hote_court);
const char *inet_ntop(int famille, const void *source, char *destination, socklen_t taille);
int      inet_pton(int famille, const char *source, void *destination);
int      kill(pid_t pid, int numero_signal);
int      listen(int sockfd, int taille);
off_t    lseek(int fd, off_t offset, int point_depart);
int      lstat(const char *chemin, struct stat *tampon);
void     *memcpy(void *destination, const void *source, size_t taille);
void     *memset(void *tampon, int caractere, size_t nombre_elements);
int      mkfifo(const char *nomFichier, mode_t mode);
int      msgctl(key_t cle, int commande, struct msqid_ds *buf);
int      msgget(key_t cle, int options);
int      msgrcv(int msqid, struct msgbuf *msg, size_t taille, long type, int options);
int      msgsnd(int msqid, struct msgbuf *msg, size_t taille, int options);
uint32_t ntohl(uint32_t reseau_long);
uint16_t ntohs(uint16_t reseau_court);
int      open(const char *chemin, int options, mode_t mode);
int      pause(void);
int      pipe(int tube[2]);
ssize_t  recvfrom(int sockfd, void *message, size_t taille, int flags,
                 struct sockaddr *adresse_source, socklen_t *taille_adresse);
ssize_t  read(int fd, void *tampon, size_t taille);
int      semctl(int semid, int semnum, int commande, union senum args);
int      semget(key_t cle, int nbSems, int options);
int      semop(int semid, struct sembuf *sops, unsigned nsops);
ssize_t  sendto(int sockfd, const void *message, size_t taille, int flags,
                 const struct sockaddr *adresse_dest, socklen_t taille_adresse);
void     *shmat(int shmid, const void *shmaddr, int options);
int      shmctl(key_t cle, int commande, struct shmid_ds *buf);
int      shmdt(const void *shmaddr);
int      shmget(key_t cle, int taille, int options);
int      sigaction(int num, const struct sigaction *action, struct sigaction *old_action);
int      sigaddset(sigset_t *ensemble, int numero_signal);
int      sigdelset(sigset_t *ensemble, int numero_signal);
int      sigemptyset(sigset_t *ensemble);
int      sigfillset(sigset_t *ensemble);
int      sigismember(sigset_t *ensemble, int numero_signal);
int      sigpending(sigset_t *ensemble);
int      sigprocmask(int comment, const sigset_t *ensemble, sigset_t *ancien);
int      sigqueue(pid_t pid, int numero_signal, const union sigval valeur);
int      sigsuspend(const sigset_t *ensemble);
int      sigwaitinfo(const sigset_t *ensemble, siginfo_t *info);
int      sigtimedwait(const sigset_t *set, siginfo_t *info, const struct timespec *timeout);
unsigned int sleep(unsigned int nombre_secondes);

```

```

int      socket(int domaine, int type, int protocole);
int      socketpair(int domaine, int type, int protocol, int sv[2]);
int      unlink(const char *nomFichier);
pid_t    wait(int *statut);
pid_t    waitpid(pid_t pid, int *statut, int options);
ssize_t  write(int fd, const void *tampon, size_t taille);

```

Constantes/macros associées à des paramètres ou des champs de structures

```

cle (msgget, shmget, semget) : IPC_PRIVATE
commande (fcntl) : F_GETFL, F_SETFL
commande (msgctl, shmctl) : IPC_RMID, IPC_STAT, IPC_SET
commande (semctl) : IPC_RMID, IPC_STAT, IPC_SET, IPC_GETVAL, IPC_GETALL, IPC_SETVAL, IPC_SETALL
comment (sigprocmask) : SIG_BLOCK, SIG_UNBLOCK, SIG_SET
domaine (socket, socketpair) : AF_LOCAL, AF_UNIX, AF_INET, AF_INET6, AF_PACKET
famille (inet_pton, inet_ntop) : AF_INET, AF_INET6
mode (mkfifo) : O_RDONLY, O_WRONLY, O_CREAT, O_EXCL, S_IRWXU, S_IRUSR, S_IWUSR, S_IXUSR...
mode (open) : S_IRWXU, S_IRUSR, S_IWUSR, S_IXUSR...
options (fcntl, open) : O_RDONLY, O_WRONLY, O_RDWR, O_CREAT, O_EXCL, O_TRUNC, O_APPEND
options (msgget, semget, shmget) : IPC_CREAT, IPC_EXCL, S_IRWXU, S_IRUSR, S_IWUSR, S_IXUSR...
options (msgrcv) : IPC_NOWAIT, MSG_EXCEPT
options (msgsnd) : IPC_NOWAIT, MSG_NOERROR
options (shmat) : SHM_RDONLY, SHM_RND
options (waitpid) : WNOHANG, WUNTRACED
point_depart (lseek) : SEEK_SET, SEEK_CUR, SEEK_END
protocole (socket, socketpair) : IPPROTO_TCP, IPPROTO_UDP
sa_flags (champ de struct sigaction) : SA_ONESHOT, SA_NOMASK, SA_SIGINFO
sa_handler (champ de struct sigaction) : SIG_IGN, SIG_DFL
sem_flag (champ de struct sembuf) : IPC_NOWAIT, SEM_UNDO
taille (inet_ntop) : INET_ADDRSTRLEN, INET6_ADDRSTRLEN
type (socket, socketpair) : SOCK_STREAM, SOCK_DGRAM

```

Macros sur `statut` de `wait`, `waitpid` : `WIFEXITED`, `WEXITSTATUS`, `WIFSIGNALED`, `WTERMSIG`, `WIFSTOPPED`, `WSTOPSIG`

Quelques structures C

<pre> struct sigaction { void (*sa_handler) (int); void (*sa_sigaction) (int, siginfo_t*, void*); sigset_t sa_mask; int sa_flags; }; union senum { int val; struct semid_ds *buf; unsigned short *array; }; struct siginfo_t { int si_signo; int si_code; sigval_t si_value; pid_t si_pid; ... }; struct msqid_ds { struct ipc_perm msg_perm; time_t msg_stime; time_t msg_rtime; time_t msg_ctime; msgnum_t msg_qnum; msglen_t msg_qbytes; pid_t msg_lspid; pid_t msg_lrpid; }; </pre>	<pre> struct in_addr { uint32_t s_addr; }; struct timespec { long tv_sec; long tv_nsec; }; struct semid_ds { struct ipc_perm sem_perm; time_t sem_otime; time_t sem_ctime; unsigned short sem_nsems; }; struct shmid_ds { struct ipc_perm msg_perm; size_t shm_segsz; time_t shm_atime; time_t shm_dtime; time_t shm_ctime; pid_t shm_cpid; pid_t shm_lpid; shmatt_t shm_nattch; }; struct sockaddr_in { sa_family_t sin_family; in_port_t sin_port; struct in_addr sin_addr; }; </pre>	<pre> struct sembuf { unsigned short sem_num; short sem_op; short sem_flg; }; struct sockaddr_in6 { sa_family_t sin6_family; in_port_t sin6_port; uint32_t sin6_flowinfo; struct in6_addr sin6_addr; uint32_t sin6_scope_id; }; struct sockaddr_un { sa_family_t sun_family; char sun_path[108]; }; struct in_addr6 { unsigned char s_addr6[16]; }; union sig_val_t { int sival_int; void *sival_ptr; }; struct timeval { long tv_sec; long tv_usec; }; </pre>
---	---	--